

PROYECTO 1



Laura Nagamine

Sofia Ku

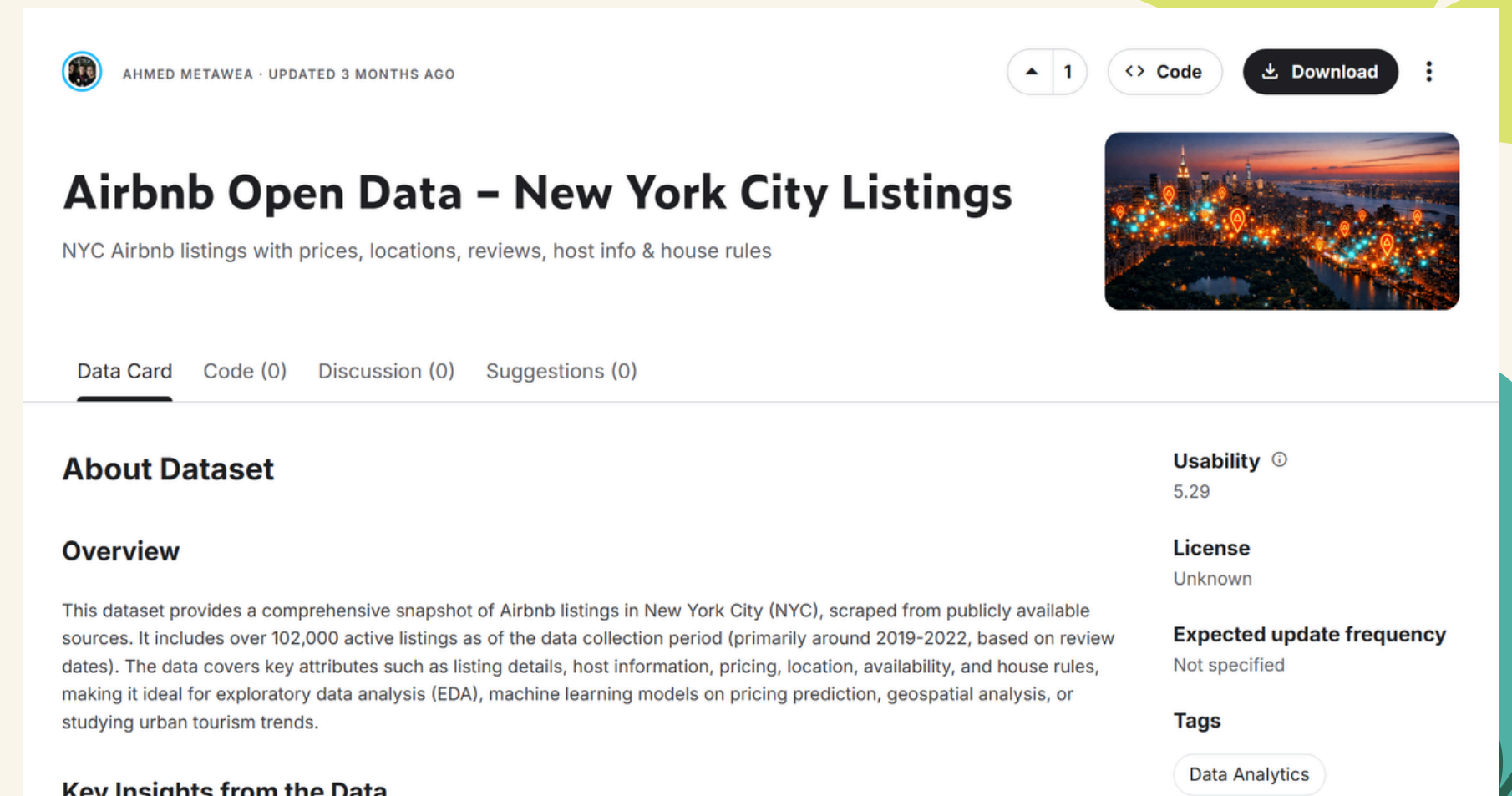
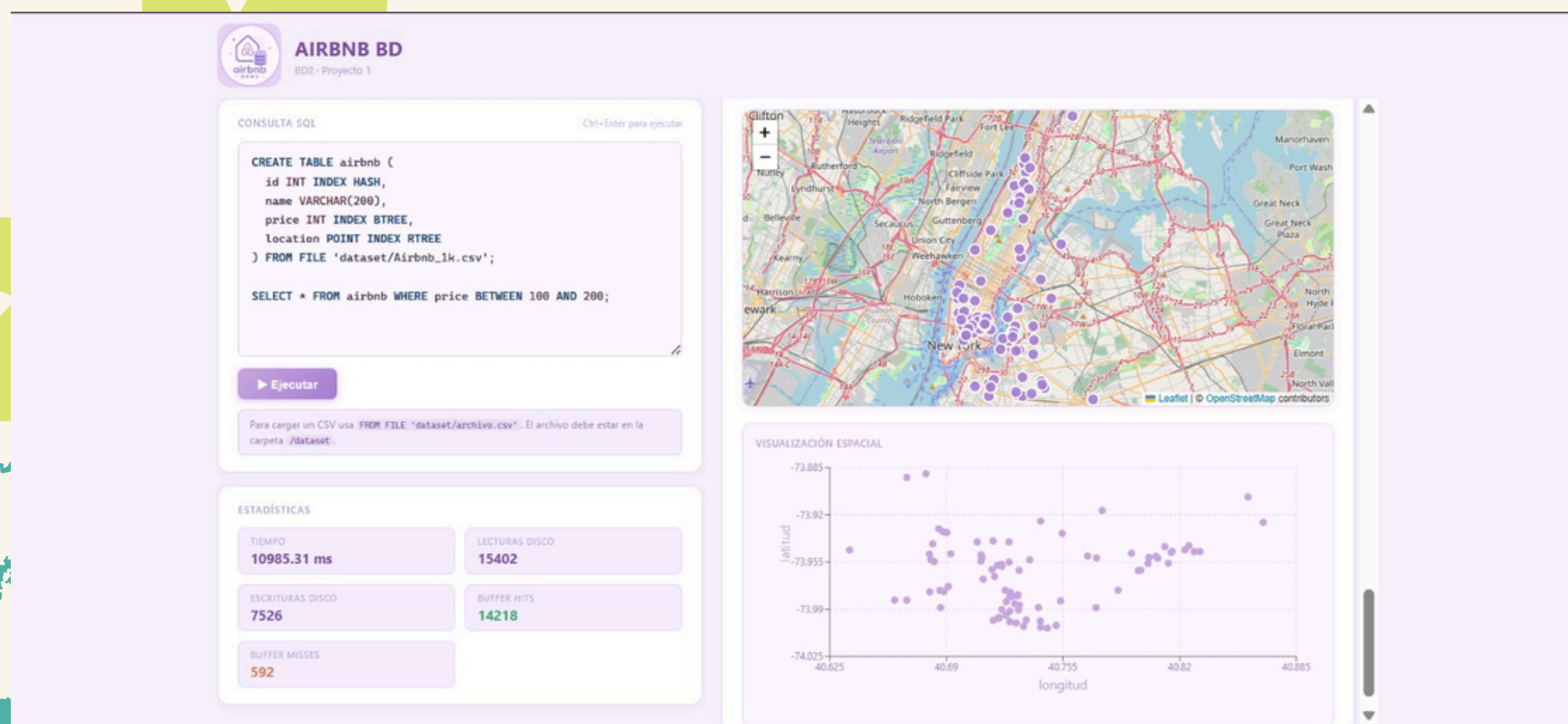
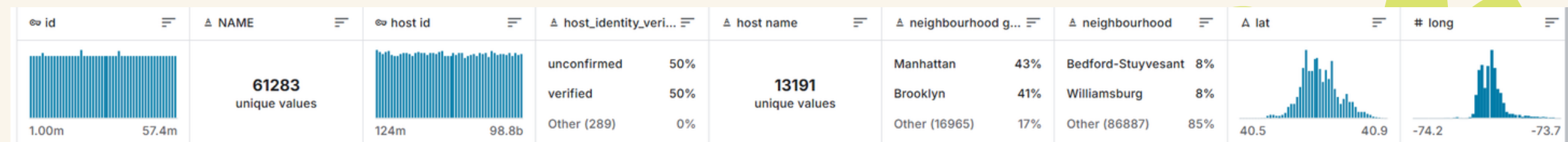
Anthony Romero

Hanks Vargas

Maria Tamayo



Introducción



<https://www.kaggle.com/datasets/ahmed1metaweaa/airbnb-open-data-new-york-city-listings-20192022>

Gramática

Usamos formato EBNF:

```
program ::= { statement ';' }
```

```
statement ::=
```

```
  'CREATE' 'TABLE' IDENTIFIER
```

```
    '(' column_def { ';' column_def } ')'
```

```
    [ 'FROM' 'FILE' PATH ]
```

```
  | 'SELECT' '*' 'FROM' IDENTIFIER
```

```
    [ 'JOIN' IDENTIFIER 'ON' column_ref '=' column_ref ]
```

```
    [ 'WHERE' condition ]
```

```
    [ 'ORDER' 'BY' IDENTIFIER [ 'ASC' | 'DESC' ] ]
```

```
  | 'INSERT' 'INTO' IDENTIFIER 'VALUES' '(' value { ';' value } ')'
```

```
  | 'DELETE' 'FROM' IDENTIFIER 'WHERE' IDENTIFIER '=' value
```

```
column_def ::=
```

```
  IDENTIFIER data_type [ 'INDEX' index_type ]
```

```
data_type ::=
```

```
  'INT' | 'FLOAT' | 'VARCHAR' '(' INT ')' | 'BOOL' | 'POINT'
```

```
index_type ::=
```

```
  'SEQUENTIAL' | 'HASH' | 'BTREE' | 'RTREE'
```

```
condition ::=
```

```
  IDENTIFIER '=' value
```

```
  | IDENTIFIER 'BETWEEN' value 'AND' value
```

```
  | IDENTIFIER 'IN' '(' point ';' 'RADIUS' NUMBER ')'
```

```
  | IDENTIFIER 'IN' '(' point ';' 'K' INT ')'
```

```
column_ref ::=
```

```
  IDENTIFIER [ '.' IDENTIFIER ]
```

```
value ::=
```

```
  INT | FLOAT | STRING | BOOLEAN | point
```

```
point ::=
```

```
  'POINT' '(' NUMBER ';' NUMBER ')'
```

```
IDENTIFIER ::= LETTER { LETTER | DIGIT | '_' }
```

```
INT ::= [ '-' ] DIGIT { DIGIT }
```

```
FLOAT ::= [ '-' ] DIGIT { DIGIT } '.' DIGIT { DIGIT }
```

```
NUMBER ::= INT | FLOAT
```

```
STRING ::= '"' { CHAR } '"'
```

```
BOOLEAN ::= 'TRUE' | 'FALSE'
```

```
PATH ::= '"' { CHAR } '"'
```

```
LETTER ::= 'a' | ... | 'z' | 'A' | ... | 'Z'
```

```
DIGIT ::= '0' | ... | '9'
```

```
CHAR ::= cualquier carácter excepto '"'
```


Scanner & Parser

Current state	Input	Next state	Acción / Token Emitido
INICIO	espacio / tab / \n	INICIO	skip
INICIO	--	COMENTARIO	skip hasta \n
INICIO	(, ; = *	INICIO	emitir símbolo correspondiente
INICIO	'	STRING	avanzar
INICIO	dígito o - seguido de dígito	NUMERO	acumular
INICIO	letra o _	IDENT	acumular
INICIO	EOF	—	emitir EOF
STRING	cualquier char ≠ '	STRING	acumular en buffer
STRING	'	INICIO	emitir STRING(buffer)
STRING	EOF	ERROR	"cadena sin cerrar"
NUMERO	dígito	NUMERO	acumular
NUMERO	. seguido de dígito	NUMERO_FLOAT	acumular
NUMERO	otro	INICIO	emitir INT(buffer)
NUMERO_FLOAT	dígito	NUMERO_FLOAT	acumular
NUMERO_FLOAT	otro	INICIO	emitir FLOAT(buffer)
IDENT	letra / dígito / _	IDENT	acumular
IDENT	otro	INICIO	lookup en tabla KEYWORDS → emitir
COMENTARIO	char ≠ \n	COMENTARIO	skip
COMENTARIO	\n / EOF	INICIO	—

Implementa un parser de descenso recursivo que convierte la secuencia de tokens en un árbol de sintaxis abstracta (AST)

Soporta las siguientes queries:

SELECT * FROM <tabla>

[JOIN <tabla_derecha> ON <tabla>.<col1> = <tabla2>.<col2>]

[WHERE <condición>]

[ORDER BY <columna> [ASC | DESC]];

CREATE TABLE <nombre_tabla> (

<columna> <tipo_dato> [INDEX <tipo_indice>],

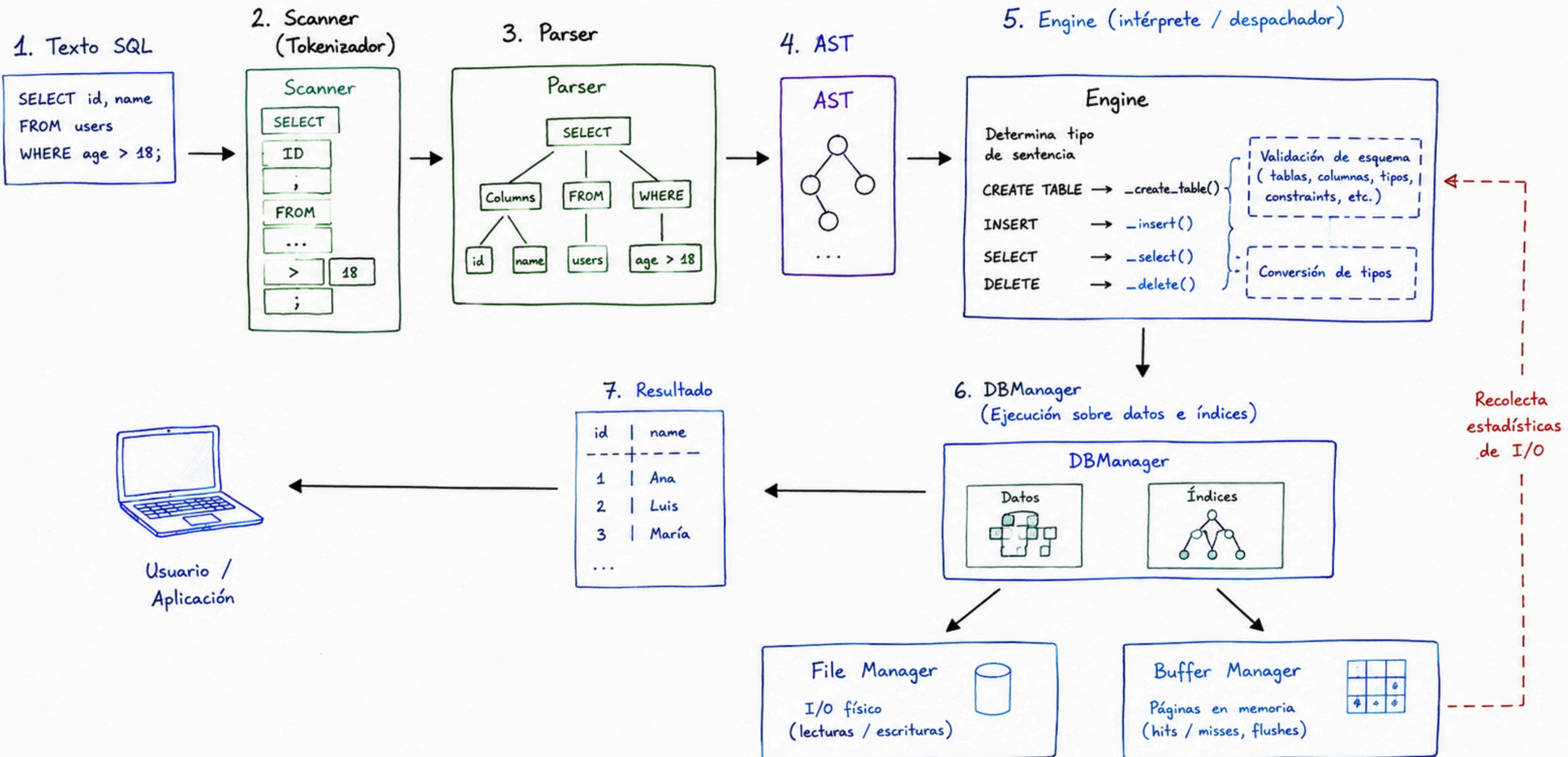
...

) [FROM FILE "<ruta_archivo>"];

INSERT INTO <nombre_tabla> VALUES (<valor1>, <valor2>, ...);

DELETE FROM <nombre_tabla> WHERE <columna> = <valor>;

engine.py y db_manager.py



SequentialFile

SEQUENTIAL FILE

IDEA GENERAL

Los registros se mantienen ordenados por clave en el archivo principal (.bin).

Las inserciones nuevas van al archivo auxiliar (.aux) sin ordenar.

Cuando el auxiliar llega a K_MAX_AUX registros, se hace un rebuild.

ARCHIVO PRINCIPAL (.bin) (ordenado por clave)

inicio →

10.50	...	←
23.10	...	←
45.00	...	←
70.20	...	←
88.80	...	←
...	...	←

puntero
al siguiente
registro

ARCHIVO AUXILIAR (.aux) (inserciones recientes, sin ordenar)

37.70	...
15.20	...
99.90	...
...	...

Cuando cantidad de
registros en .aux
≥ K_MAX_AUX

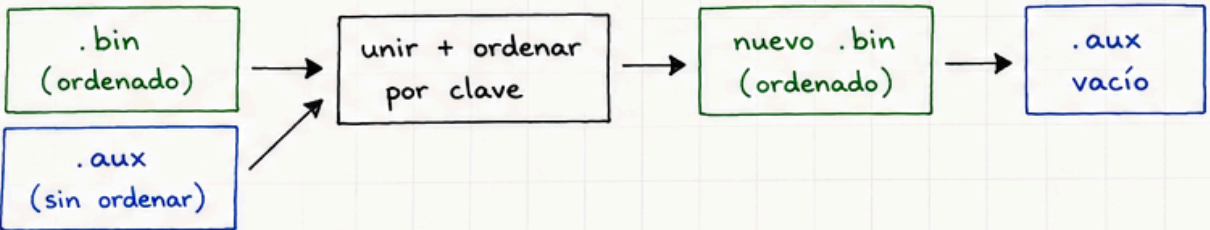
⇒ **REBUILD**

FORMATO DE REGISTRO (240 bytes)



TAMAÑO DE PÁGINA: 4096 bytes

REBUILD (fusión y reordenamiento)



NOTAS

- next_offset es el índice del siguiente registro (-1 = fin)
- deleted = 1 indica eliminación lógica

OPERACIONES

add(key, data):

- se agrega al final del .aux
- si count(.aux) ≥ K_MAX_AUX ⇒ rebuild()

search(key):

- 1) búsqueda binaria en .bin
- 2) si no está, búsqueda lineal en .aux

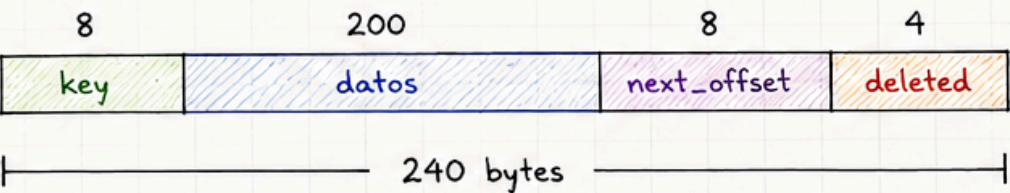
range_search(low, high):

- binaria para encontrar el inicio en .bin
- y luego recorrido secuencial + revisión del .aux

remove(key):

- se marca deleted = 1 (no se elimina físicamente)

EJEMPLO DE UN REGISTRO (bytes)



Campo	Tipo de Dato	Tamaño	Descripción
Key	f64 (double)	8 bytes	Clave de búsqueda
Data	bytes	200 bytes	Información asociada
Next	i64 (long)	8 bytes	Puntero lógico al siguiente
Deleted	i32 (int)	4 bytes	Flag de borrado (0 =

Extendible Hashing

Directory Global
(index.dat)

D=2

00
01
10
11

Buckets
(data.dat)

0	10, 34, 6, 12
1	13, 23

index.dat (HEADER + DIRECTORY)

• **HEADER** (fijo: 24 bytes)

4 bytes	4 bytes	4 bytes	4 bytes	4 bytes	4 bytes
global_depth (int32)	directory_size (int32)	free_head (int32)	bucket_count (int32)	fb (int32)	reserved (int32)
24 bytes					

global_depth: profundidad global (d)
 directory_size: $= 2^d$ (cantidad de entradas)
 free_head: índice del primer bucket libre
 bucket_count: cantidad total de buckets creados
 fb: factor de bucket (capacidad máxima)
 reserved: reservado para futuro uso

• **DIRECTORY** (variable: directory_size entradas)

directory_size = 2 ^d entradas								
4 bytes	2 bytes	4 bytes	2 bytes	4 bytes	2 bytes	...	4 bytes	2 bytes
page_id (int32)	slot_id (int16)	page_id (int32)	slot_id (int16)	page_id (int32)	slot_id (int16)	...	page_id (int32)	slot_id (int16)
6 * directory_size bytes								

Cada entrada del directorio:

- page_id : página donde está el bucket
- slot_id : slot del bucket en esa página

data.dat (BUCKET)

2 bytes	2 bytes	fb * 8 bytes	fb * 8 bytes	4 bytes	4 bytes
local_depth (int16)	count (int16)	keys[] (fb claves, cada una 8 bytes) (ej. float64)	values[] (fb valores, cada uno 8 bytes) (ej. record_id int64)	next (int32)	free_list (int32)
4 + (fb*8) + (fb*8) + 4 + 4 bytes = 16 + (fb * 16) bytes					

local_depth: profundidad local del bucket
 count: cantidad de claves almacenadas
 keys[]: claves almacenadas
 values[]: valores asociados (record_id)
 next: puntero al siguiente bucket (overflow)
 free_list: puntero al siguiente bucket libre

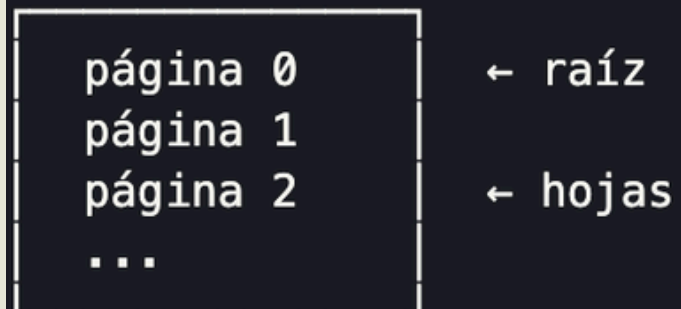


B+ Tree

Estructuras de Indexación sobre
Memoria Secundaria

Modelo de almacenamiento físico

archivo: tabla_col.bpt

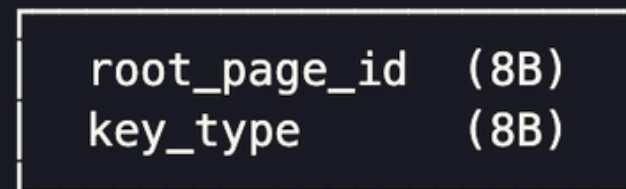


PAGE_SIZE = 4096 bytes

Capacidad por página (INT, tamaño 8B):

- Hoja: $(4096 - \text{header}) \div (8 + 200) \approx 19$ entradas
- Interna: $(4096 - \text{header} - 8) \div (8 + 8) \approx 255$ punteros

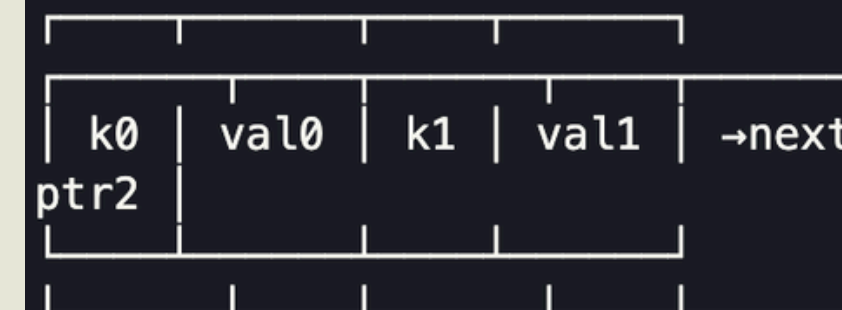
archivo: tabla_col.meta



Estructura de un nodo

HEADER (9 bytes): is_leaf (1B) | n_keys (2B) | next_leaf (8B, solo hojas)

Nodo HOJA:



key: 8B

value: 200B

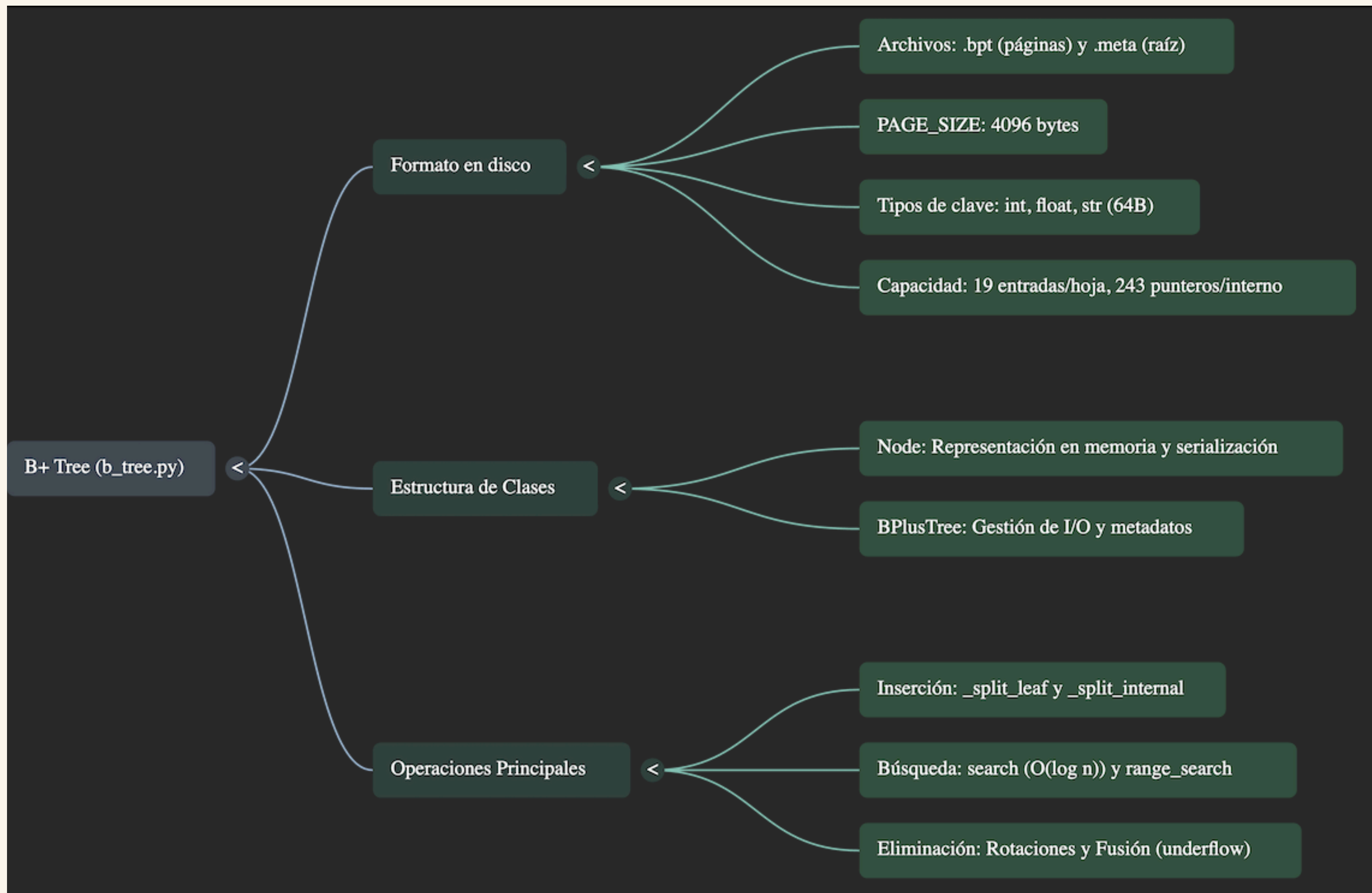
Nodo INTERNO:



ptr: 8B

key: 8B

val = record_id empaquetado en bytes (200B)



Operaciones Búsqueda e inserción

```
search(key):
  nodo ← raíz
  mientras no es_hoja:
    i ← primer i donde key < keys[i]
    nivel ← 0(log n) por
    nodo ← leer(children[i])
  retornar values[i] si keys[i] == key

add(key, value):
  _insert(raíz, key, value) ← recursivo top-down
  si hoja → insertar ordenado
  si overflow → _split_leaf() → propagar clave al
  padre
  si interno → descender al hijo correcto
  si hijo se dividió → absorber nueva clave
  si overflow → _split_internal()
  si raíz se divide → crear nueva raíz con una sola clave
```


Operaciones

Split: hoja vs interno

SPLIT HOJA (copia la clave media):

$[1, 2, 3, 4, 5] \rightarrow [1, 2] + [3, 4, 5]$

sube al padre: 3 next_leaf actualizado ✓

SPLIT INTERNO (sube la clave media, no se copia):

$[10, 20, 30, 40] \rightarrow [10] + [30, 40]$

sube al padre: 20 la clave media "desaparece" de las hojas x

Diferencia clave:

- Hoja: la clave media PERMANECE en la hoja derecha
- Interno: la clave media SUBE al padre y no queda en ningún hijo

Operación Range search y scan_all

```
range_search(lo, hi):  
    hoja ← _find_leaf(lo)           ← baja por el árbol hasta  
    lo  
    mientras hoja existe:  
        para cada (k, v) en hoja:  
            si  $k > hi$  → retornar  
            si  $lo \leq k \leq hi$  → agregar v  
        hoja ← hoja.next_leaf       ← salta al siguiente nodo  
hoja  
  
scan_all():  
    hoja ← el nodo hoja más a la izquierda  
    recorrer toda la lista enlazada de hojas  
    retornar todos los values en orden
```


Operación remove

`remove(key):`

Bajar recursivamente hasta la hoja y eliminar.

Si `len(keys) < min_keys` → UNDERFLOW → 3 opciones:

1. Redistribuir ← hermano izquierdo tiene de sobra
rotar: mover último del izquierdo al frente del hijo
2. Redistribuir → hermano derecho tiene de sobra
rotar: mover primero del derecho al final del hijo
3. Fusionar (merge)
absorber hijo en hermano + bajar separador del padre
si padre queda con 0 claves → nueva raíz = su único hijo

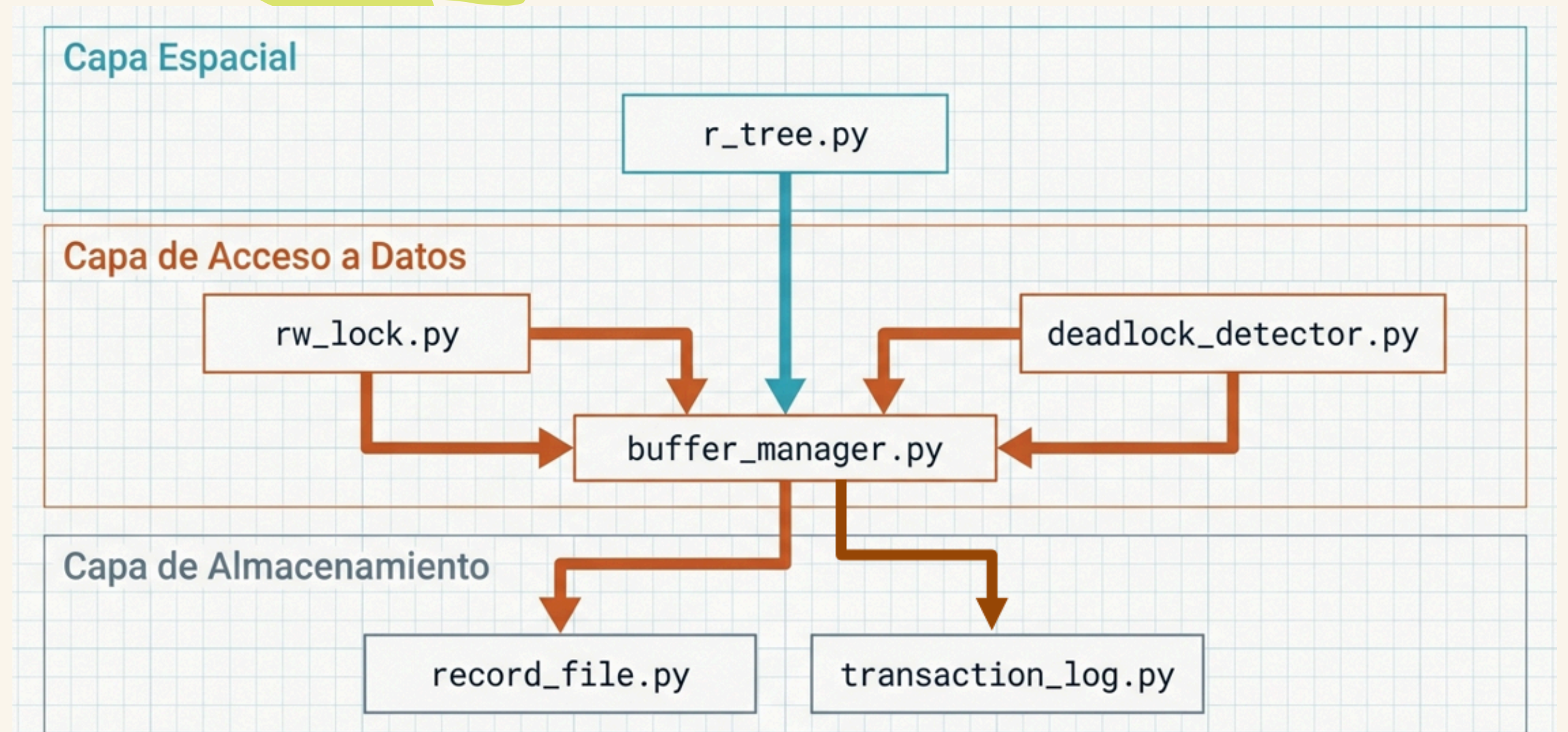


Control de Concurrencia & R-Tree

BufferManager · RWLock · DeadlockDetector · R-Tree

Capas:

El sistema no aísla los bloqueos por índice; los centraliza en el BufferManager para proteger incluso el archivo físico base



Wait-For Graph + DFS

El algoritmo DFS recorre el grafo de dependencias en tiempo real. En el instante exacto en que se cierra el ciclo, se lanza la excepción. Cero tiempos de espera arbitrarios.

BUFFERMANAGER — PUNTO CENTRAL DE CONTROL

`lock_page()`

`unlock_page()`

`read_page()`

`write_page()`

`append_page()`

`_evict()` LRU

Inserción & División de Nodos (Quadratic Split)

`add(x, y, data)`

`_choose_subtree()`

`_insert()` recursivo

Nueva raíz

Inserción & División de Nodos (Quadratic Split)

Range Search & K-NN Algoritmos de Consulta

- › `_pick_seeds()`: elige el par con mayor 'desperdicio' si estuvieran juntos
- › `_pick_next()`: asigna la entrada más 'obvia' al grupo que menos crece
- › Garantía m: si un grupo necesita todas las restantes para llegar al mínimo, se le asignan todas
- › Balance calidad/velocidad: $O(M^2)$ vs R*-Tree $O(M^3)$

knn(cx, cy, k) – Best-First

Heap de prioridad

Cola ordenada por distancia mínima al MBR. Siempre expande el nodo más prometedor.

is_result flag

Nodo interno → distancia al MBR (cota inferior). Punto hoja → distancia euclidiana exacta.

Garantía de optimalidad

Cuando un punto real es el mínimo del heap, ningún otro puede ser más cercano. Es exacto.

Ventaja vs naive

No requiere radio creciente ni múltiples pasadas. Visita el mínimo de páginas posible.

range_search(cx, cy, r)

Fase 1 — Poda MBR

Si el MBR del nodo NO intersecta el círculo → toda la rama se descarta sin leerla (`intersects_circle`)

Fase 2 — Verificación exacta

En hojas: `math.hypot(cx-x, cy-y) <= r` para cada punto candidato. Solo pasan los reales.

Truco geométrico

El punto más cercano del rectángulo al centro: `nx=clamp(cx, x_min, x_max)`. Si `dist ≤ r` → intersecta.



Experimentación

Metodología

Dataset

Airbnb Open Data / New York City (Kaggle)

Tamaños

$n = 1,000$ | $n = 10,000$ | $n = 100,000$

Indices evaluados

B+Tree / Hash / Sequential / R-Tree

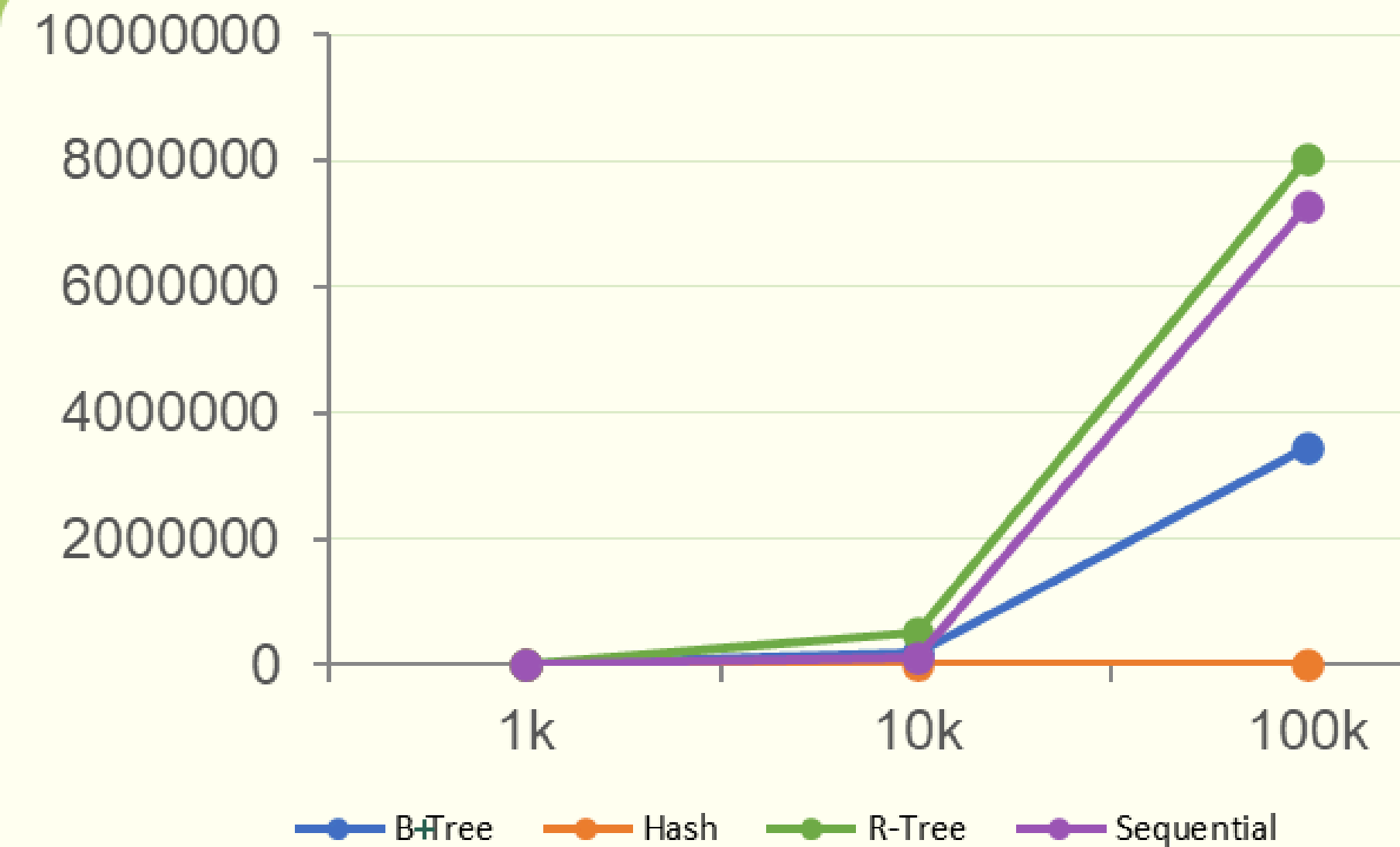
Métricas

Tiempo de ejecución (ms) • Accesos a disco (lecturas / escrituras)

Operaciones

- ▶ Inserción
- ▶ Búsqueda puntual
- ▶ Búsqueda por rango
- ▶ Eliminación
- ▶ Consultas espaciales (radio • KNN)

INSERCIÓN — TIEMPO POR ÍNDICE



AIRBNB BD

BD2 · Proyecto 1

```
CREATE TABLE airbnb_seq_100k (  
  id INT,  
  name VARCHAR(200),  
  price INT INDEX SEQUENTIAL,  
  location POINT  
) FROM FILE 'dataset/Airbnb_100k.csv';
```

Ejecutar

Para cargar un CSV usa `FROM FILE 'dataset/archivo.csv'`. El archivo debe estar en la carpeta `/dataset`.

ESTADÍSTICAS

TIEMPO
7301036.26 ms

LECTURAS DISCO
20517

ESCRITURAS DISCO
112850

BUFFER HITS
0

BUFFER MISSES
0

RESULTADOS

✓

tabla 'airbnb_seq_100k' creada (102591 filas cargadas)

Análisis

Hash es el más rápido: $O(1) \rightarrow 26,914$ ms en 100k. Mínimas escrituras (2,943) en 100k.

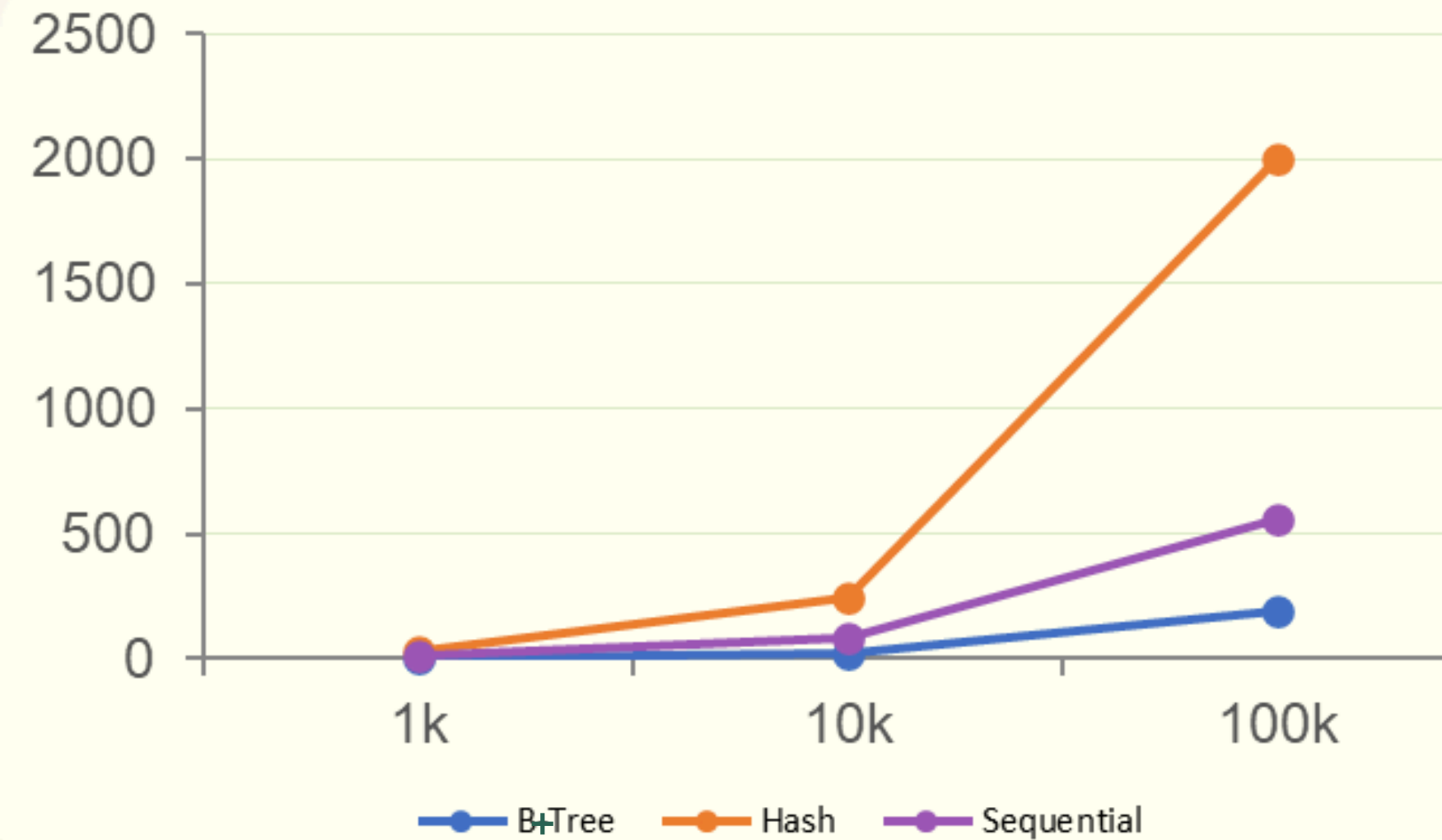
B-Tree crece de forma controlada $O(\log n)$.

R-Tree el más costoso — 8M ms y 1M lecturas en 100k.

Sequential escala linealmente $O(n)$. Menores lecturas pero muchas escrituras en 100k.



BÚSQUEDA PUNTUAL



AIRBNB BD

BD2 · Proyecto 1

```
SELECT * FROM airbnb_btree_10k WHERE price = 459;
```

Ejecutar

Para cargar un CSV usa `FROM FILE 'dataset/archivo.csv'`. El archivo debe estar en la carpeta `/dataset`.

ESTADÍSTICAS

TIEMPO 23.55 ms	LECTURAS DISCO 8
ESCRITURAS DISCO 0	BUFFER HITS 0
BUFFER MISSES 4	

RESULTADOS

ID	NAME
6332136	Cozy room in 1bedroom apt
5392121	Studio Apt East Harlem
4522801	Brooklyn Life, Easy to Manhattan (30+ Days only)
2938252	Big Sunlit Studio - Nice Bed - 18 min to Manhattan
2216396	East Village Loft - 1300 sq/ft home - 2 Bedrooms!
1466373	E Williamsburg Apartment with Yard
1389603	Oceanview,close to Manhattan
1027846	Charming Brownstone 3 - Near PRATT

MAPA

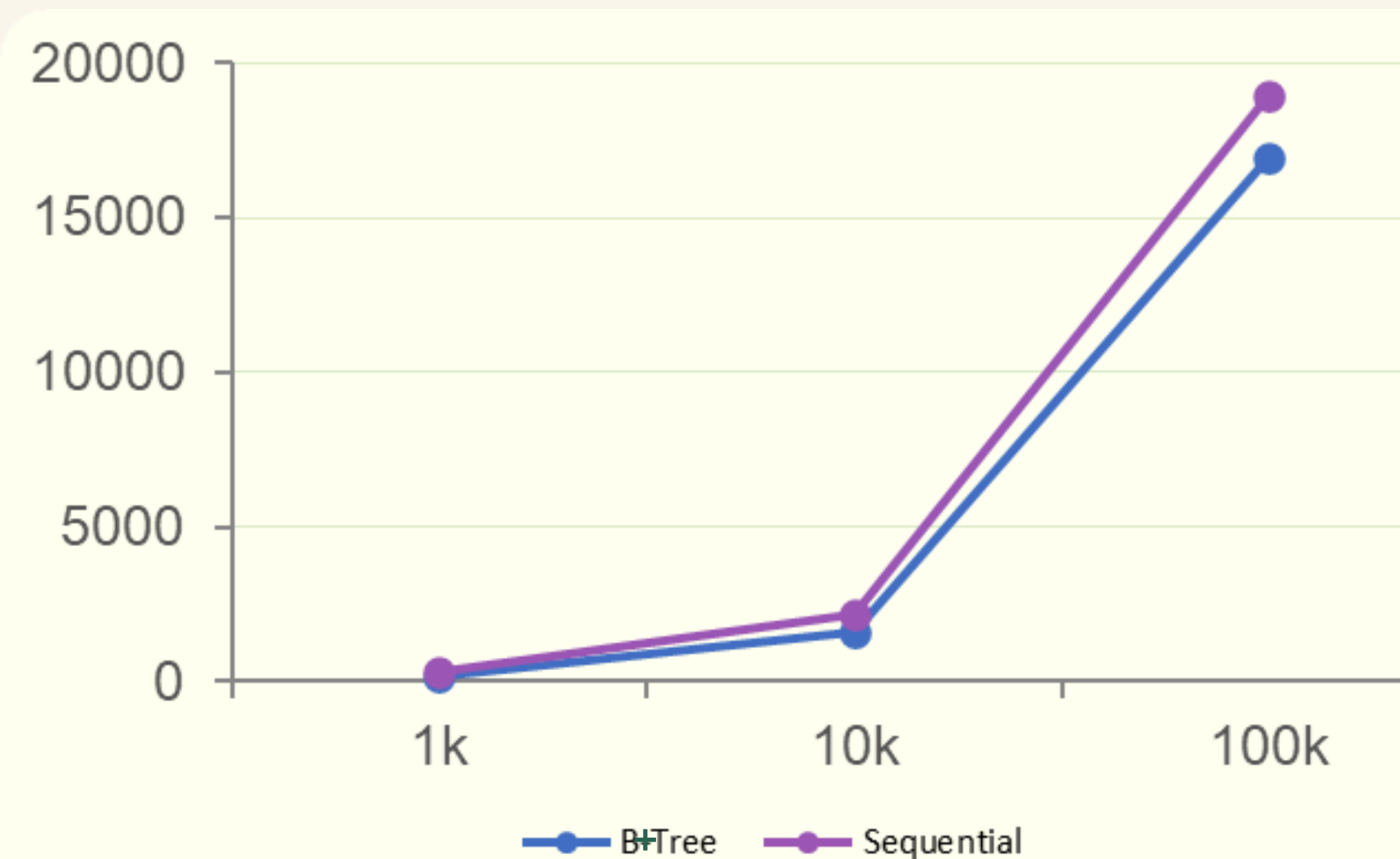
Análisis

B-Tree el más eficiente: 192 ms y solo 28 lecturas en 100k — $O(\log n)$ bien controlado.

Hash no escala bien — 2,001 ms en 100k a pesar de solo 6 lecturas de disco.

Sequential se degrada linealmente; solo 2 lecturas pero tiempo muy alto en 100k.

BÚSQUEDA POR RANGO



AIRBNB BD

BD2 · Proyecto 1

CONSULTA SQL

Ctrl+Enter para ejecutar

```
SELECT * FROM airbnb_btree_100k WHERE price BETWEEN 200 AND 600;
```

Ejecutar

Para cargar un CSV usa `FROM FILE 'dataset/archivo.csv'`. El archivo debe estar en la carpeta `/dataset`.

ESTADÍSTICAS

TIEMPO

16941.14 ms

LECTURAS DISCO

6854

ESCRITURAS DISCO

0

BUFFER HITS

0

RESULTADOS

ID	NAME
57326547	Studio in Brooklyn Heights with Private Entrance
57145945	Magical, Welcoming & Beautiful Huge 1 Bedroom Apt
57138765	Townhouse apt. Close to city. 3 bedrooms & 2 bath
57076355	*WINTER DISCOUNT WILLIAMSBURG CHIC W/PRIVATE
56835552	Friendly place for most suitable for Bangladeshis
56005998	Luxurious home in the sky overlooking 5th Ave.
55882835	Private Bath & Fire Escape Brooklyn Bedroom
55780107	New York Apartment / HIDDEN GEM ROSEDALE LLC
55464191	!AMAZING PRIVATE ROOM 2 MIN FROM TRAIN STATION
54582168	Cozy shared apt in Midtown Manhattan

Análisis

B+Tree domina en rangos gracias a sus hojas enlazadas — 16,941 ms y 6,854 lecturas en 100k.

Hash no aplica para este tipo de consultas por su naturaleza no ordenada.

Sequential mínimas lecturas (2 en 100k) pero escaneo completo lo hace significativamente más lento.

DELETE



AIRBNB BD
BD2 · Proyecto 1

```
DELETE FROM airbnb_seq_100k WHERE price = 459;
```

Ejecutar

Para cargar un CSV usa `FROM FILE 'dataset/archivo.csv'`. El archivo debe estar en la carpeta `/dataset`.

ESTADÍSTICAS

TIEMPO
2676.09 ms

LECTURAS DISCO
999

ESCRITURAS DISCO
1

BUFFER MISSES
0

BUFFER HITS
0

RESULTADOS

✓ 111 eliminado(s)

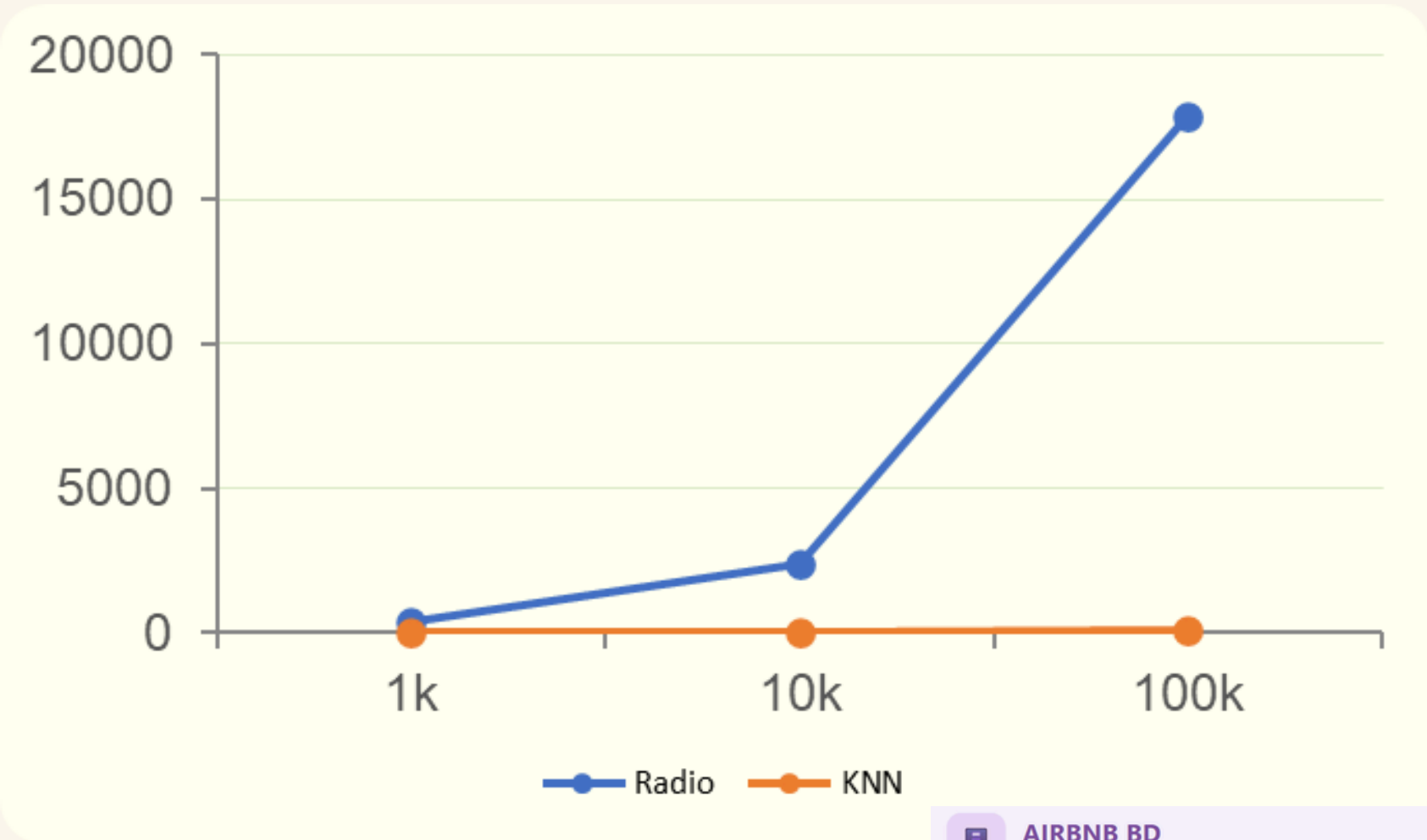
Análisis

B-Tree mejor rendimiento global — 1,751 ms en 100k con solo 181 escrituras.

Sequential mínimas escrituras (1) pero 2,676 ms en 100k — ineficiente a escala.

Hash comportamiento inconsistente en tiempo; pocas escrituras en 10k y 100k.

R-TREE — CONSULTAS ESPACIALES



```
SELECT * FROM airbnb_rtree_100k
WHERE location IN (POINT(40.7580, -73.9855), K 10);
```

Ejecutar

Para cargar un CSV usa `FROM FILE 'dataset/archivo.csv'`. El archivo debe estar en la carpeta `/dataset`.

ESTADÍSTICAS

TIEMPO
94.01 ms

LECTURAS DISCO
44

ESCRITURAS DISCO
0

BUFFER HITS
0

ID	NAME	PRICE	LOCATION	_DIST
36872669	1 Floor - 7 Spacious Rooms in Times Square	433	(40.75774 , -73.98552)	0.000
29264736	Walk distance to Broadway/Bryant Park/Top of rock	101	(40.75755 , -73.98573)	0.000
21180721	Entire suite with breathtaking views of NYC	1040	(40.75866 , -73.98535)	0.000
55938617	Entire suite with breathtaking views of NYC	1040	(40.75866 , -73.98535)	0.000
25796292	Quiet Room Next to Times Square and Bryant Park	461	(40.75745 , -73.98596)	0.000
54590453	Studio Penthouse	617	(40.75762 , -73.98486)	0.000
19832557	Studio Penthouse	617	(40.75762 , -73.98486)	0.000
37579613	AMAZING Times Square Room/ 44th St./ Broadway	583	(40.75812 , -73.98463)	0.000
31227058	Luxury Times Square Suite, 24 /7 Onsite Team	929	(40.7572 , -73.98509)	0.000
5537928	Times Square room with private bath	150	(40.75731 , -73.98479)	0.001

AIRBNB BD

BD2 - Proyecto 1

CONSULTA SQL

Ctrl+Enter para ejecutar

```
SELECT * FROM airbnb_rtree_100k
WHERE location IN (POINT(40.7580, -73.9855), RADIUS 0.05);
```

Ejecutar

Para cargar un CSV usa `FROM FILE 'dataset/archivo.csv'`. El archivo debe estar en la carpeta `/dataset`.

ESTADÍSTICAS

TIEMPO
17872.67 ms

LECTURAS DISCO
5598

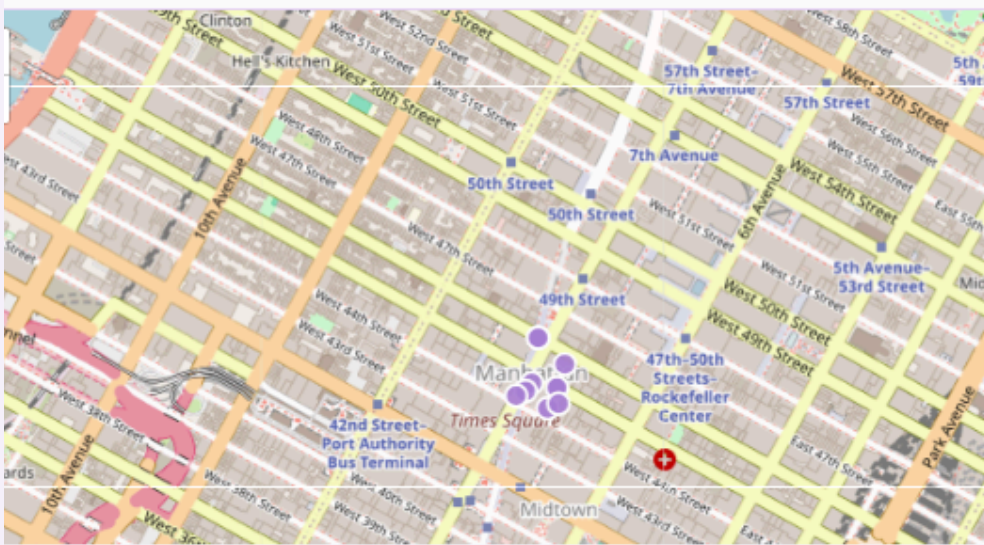
ESCRITURAS DISCO
0

BUFFER HITS
0

MAPA

Solo RTree

VISUALIZACIÓN ESPACIAL



CONCLUSIÓN EXPERIMENTAL

Índice	Complejidad	Mejor operación	Resultado observado
Hash	$O(1)$	Inserción	Rápido en inserción, pobre en búsquedas
B-Tree+	$O(\log n)$	Búsqueda · Rango · DELETE	Estable y balanceado — más versátil
Sequential	$O(n)$	—	No escala — solo datasets pequeños
R-Tree	$O(\log n)$ espacial	KNN · Radio	Eficiente en KNN (crecimiento sublineal)

Conclusión

La elección del índice depende del patrón de operaciones de la aplicación. No existe una estructura universal.

- B-Tree+** es la opción más versátil para cargas mixtas.
- **Hash** sobresale en inserciones masivas.
 - **R-Tree** es irremplazable para datos geoespaciales.

ALGORITMOS DE
MEMORIA
EXTERNA
(BONUS PARCIAL)

Replacement Selection

Algoritmo de memoria externa

CLASES IMPLEMENTADAS

ReplacementSelection

_RunReader

_RunWriter

FUNCIÓN `generate_runs()`

1 Llenar heap

2 Pop mínimo

3 Comparar siguiente

4 Nuevo run

RESULTADOS VERIFICADOS

N = 1,000

1.47x

N = 10,000

1.96x

N = 100,000

2.00x

USO EN SQL

```
SELECT * FROM airbnb  
ORDER BY price ASC; --replacement
```


External Sort

EXTERNAL SORT (ORDENAMIENTO EXTERNO)

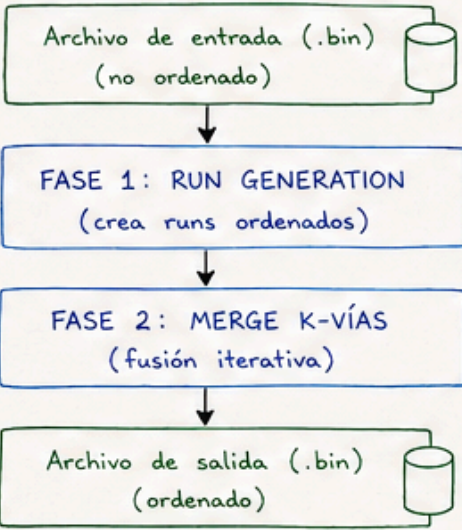
IDEA GENERAL

Algoritmo para ordenar archivos que no caben en memoria principal.

Trabaja en dos grandes fases:

- 1) Genera RUNS ordenados en disco
- 2) Fusiona los RUNS hasta obtener un único archivo ordenado.

FLUJO GENERAL DEL ALGORITMO



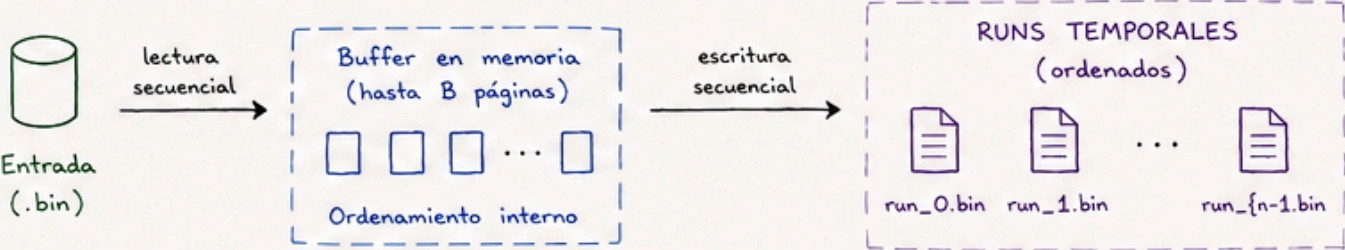
FORMATO DE REGISTRO (tamaño fijo)

8 bytes	200 bytes	8 bytes	4 bytes
key (float64)	datos (200 bytes)	next_offset (int64)	deleted (int32)
TAMAÑO TOTAL POR REGISTRO: 240 bytes			

TAMAÑO DE PÁGINA: 4096 bytes

FASE 1: RUN GENERATION (creación de runs)

1. Se leen hasta B páginas (del buffer) del archivo de entrada.
2. Los registros se cargan en memoria y se ordenan (merge sort interno).
3. Se escribe el run ordenado en un archivo temporal (run_i.bin).
4. Repetir hasta procesar todo el archivo de entrada.

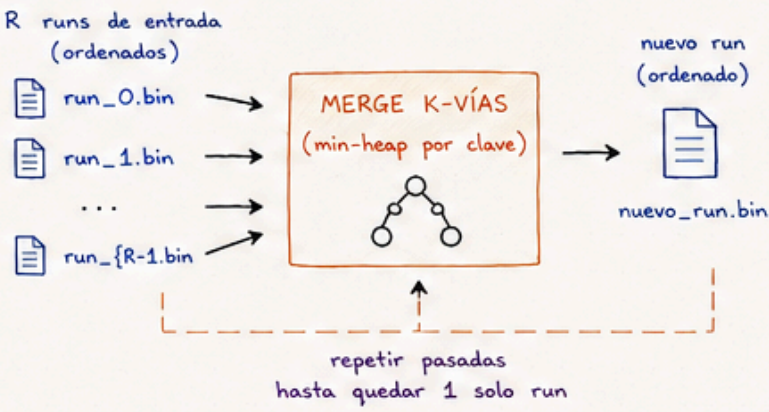


B: páginas de buffer disponibles
($B \geq 3$)

$R = B - 1$: número máximo de runs que se pueden fusionar por pasada (fan-in)

FASE 2: MERGE K-VÍAS (fusión iterativa)

1. Se toman hasta R runs ordenados.
2. Se fusionan usando un min-heap (por clave).
3. El resultado se escribe en un nuevo run temporal.
4. Repetir en pasadas sucesivas hasta quedar un único run.

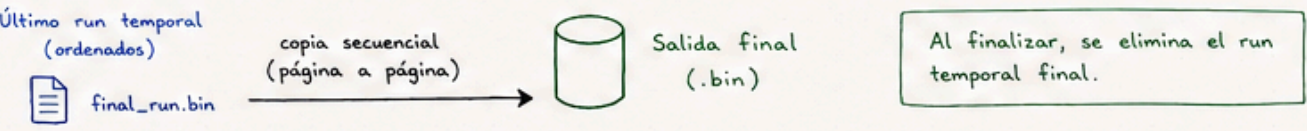


EJEMPLO ($B = 8$)

$R = B - 1 = 7$

Se fusionan hasta 7 runs por pasada.

COPIA FINAL AL DESTINO



ARCHIVOS QUE INTERACTÚAN

- Entrada (.bin): archivo original sin ordenar. Fuente de datos.
- Salida (.bin): archivo final con todos los registros ordenados.
- Runs temporales (run_*.bin): archivos secuenciales con runs ordenados generados en la Fase 1 y en cada pasada de merge.
- Archivo de control / metadatos (.ctl): (opcional) guarda estadísticas del sort, parámetros, cantidad de runs, etc.

ESTADÍSTICAS DE I/O (recolectadas)

- Lecturas físicas (File Manager)
- Escrituras físicas (File Manager)
- Páginas leídas / escritas (Buffer Manager)
- Tiempo total de ordenamiento
- Pasadas realizadas
- Cantidad de runs generados

PROPIEDADES

- ✓ Complejidad: $O(N \log_k(N/M))$ donde:
 N = # registros
 M = memoria disponible (en registros)
 k = fan-in ($= B - 1$)
- ✓ Requiere múltiples pasadas sobre disco (I/O intensivo)
- ✓ Usa archivos secuenciales $\rightarrow$ lecturas y escrituras eficientes

LEYENDA: Archivo permanente (entrada/salida) Memoria / Buffer Archivo temporal (runs) Archivo de control / metadatos

Ejemplo de consulta:
`SELECT * FROM ... [WHERE condición]`
`[ORDER BY condición]`

EXTERNAL HASHING JOIN

CLIENTES (tabla más pequeña - BUILD)

n = 4 {

id	nombre
1	Ana
2	Luis
3	María
4	Carlos

FASE 1: BUILD (particionado externo)

Se recorre CLIENTES y cada fila se escribe en el bucket correspondiente usando $\text{hash}(\text{id}) \% B$.
Cada bucket se materializa en disco (archivos .jsonl).



PARTICIONES EXTERNAS (buckets en disco)

B = 4 buckets

0	bucket_0.jsonl [(4, Carlos)]	claves: 4
1	bucket_1.jsonl [(1, Ana)]	claves: 1
2	bucket_2.jsonl [(2, Luis)]	claves: 2
3	bucket_3.jsonl [(3, María)]	claves: 3

VENTAS (tabla más grande - PROBE)

m = 6 {

venta_id	clientes_id	monto
101	2	100.00
102	1	250.00
103	2	75.00
104	5	300.00
105	3	120.00
106	1	80.00

FASE 2: PROBE (búsqueda por bucket)

- Se recorre VENTAS y para cada fila:
- ① se calcula $\text{bucket} = \text{hash}(\text{clientes_id}) \% B$
 - ② se lee solo ese archivo `bucket_i.jsonl`
 - ③ se buscan coincidencias comparando la clave real (`clientes_id`).
 - ④ si existe, se combina y se produce el resultado.



RESULTADO DEL JOIN

venta_id	clientes_id	monto	id	nombre
101	2	100.00	2	Luis
102	1	250.00	1	Ana
103	2	75.00	2	Luis
105	3	120.00	3	María
106	1	80.00	1	Ana

La venta 104 (`clientes_id = 5`) **no produce resultado** porque no existe la clave 5 en CLIENTES.

Función hash:

$\text{bucket} = \text{hash}(\text{clave}) \% B$
(usa SHA-256)



GROUP BY - COUNT(*)

Cambios en Scanner y Parser

SCANNER – 2 tokens nuevos:

GROUP = auto()

COUNT = auto()

PARSER – SELECT antes vs ahora:

Antes: SELECT * FROM tabla

→ solo admitía STAR

Ahora: SELECT col, COUNT(*) FROM tabla

→ lista de expresiones

AST generado:

```
{
  "type": "SELECT",
  "columns": [{"kind": "COLUMN", "name": "col"},
{"kind": "COUNT_STAR"}],
  "group_by": "col"
}
```

Lógica de agrupación en el Engine

```
_apply_group_by(rows, columns, group_col):  
  
    rows = [{"cat":"A","val":10}, {"cat":"B","val":5},  
            {"cat":"A","val":3}]  
  
    groups = {}  
    for row in rows:  
        key = row[group_col]           → "A", "B", "A"  
        groups[key] += 1  
  
    → [{"cat":"A", "COUNT(*)":2},  
        {"cat":"B", "COUNT(*)":1}]  
  
Complejidad: O(n) tiempo · O(k) espacio (k = grupos distintos)
```

Lógica de agrupación en el Engine

```
_apply_group_by(rows, columns, group_col):  
  
    rows = [{"cat":"A","val":10}, {"cat":"B","val":5},  
            {"cat":"A","val":3}]  
  
    groups = {}  
    for row in rows:  
        key = row[group_col]           → "A", "B", "A"  
        groups[key] += 1  
  
    → [{"cat":"A", "COUNT(*)":2},  
        {"cat":"B", "COUNT(*)":1}]  
  
Complejidad: O(n) tiempo · O(k) espacio (k = grupos distintos)
```


DEMO frontend



AIRBNB BD

BD2 - Proyecto 1

CONSULTA SQL

Ctrl+Enter para ejecutar

```
CREATE TABLE airbnb (  
  id INT INDEX HASH,  
  name VARCHAR(200),  
  price INT INDEX BTREE,  
  location POINT INDEX RTREE  
) FROM FILE 'dataset/Airbnb_1k.csv';  
  
SELECT * FROM airbnb WHERE price BETWEEN 100 AND 200;
```

Ejecutar

Para cargar un CSV usa FROM FILE 'dataset/archivo.csv'. El archivo debe estar en la carpeta /dataset.

ESTADÍSTICAS

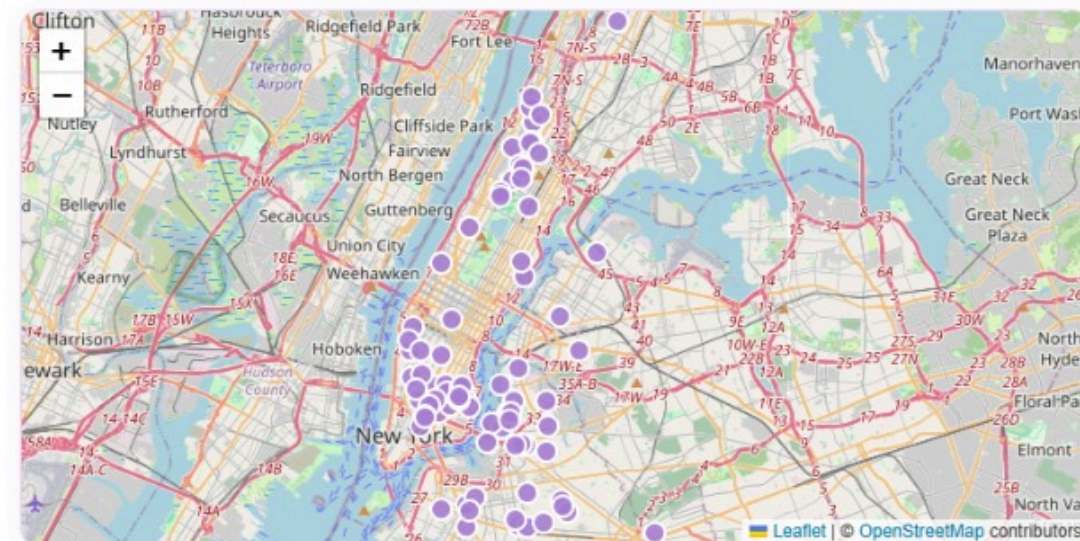
TIEMPO
10985.31 ms

LECTURAS DISCO
15402

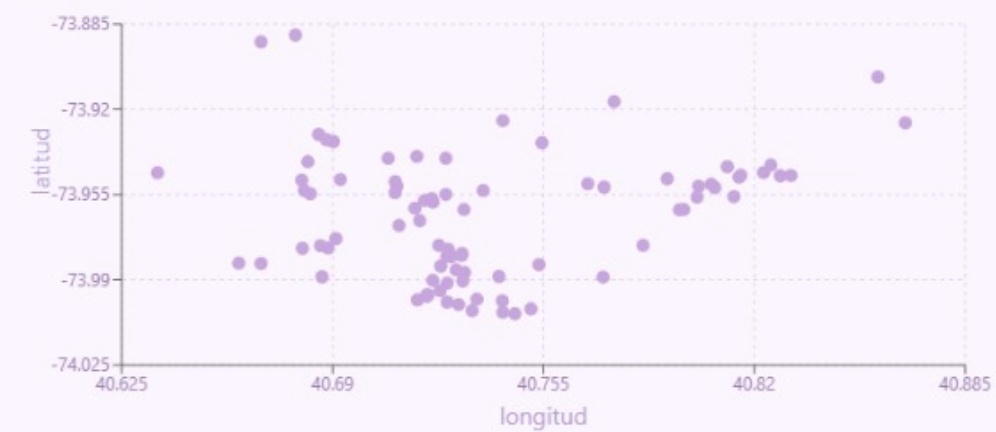
ESCRITURAS DISCO
7526

BUFFER HITS
14218

BUFFER MISSES
592



VISUALIZACIÓN ESPACIAL





MUCHAS
GRACIAS